IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

Project Garuda: A Privacy-First Edge-AI Home Security System on Raspberry Pi 5 with Hailo-8L NPU

GONUGONDLA VEERA MANIKANTA[1] AND SWATHI TANGI[1]

[1]Department of Electrical and Electronics Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, Karnataka, India (e-mail: manikantagonugondla@gmail.com; swathi.tangi@manipal.edu)

Corresponding author: Gonugondla Veera Manikanta (e-mail: manikantagonugondla@gmail.com).

This work was carried out as an independent project. All hardware, software, and cloud services were self-funded.

ABSTRACT Consumer home security is stuck between cloud-backed cameras that upload every frame and charge a subscription, and bare-bones DIY builds that rarely go beyond motion detection. This paper presents Project Garuda, a privacy-first security system built on a Raspberry Pi 5 and a Hailo-8L neural processing unit (13 TOPS, INT8) for under US$500. A YOLOv8s detector, fine-tuned on 4,982 images covering four threat classes (Hammer, Knife, Person, Scissors), sustains 52.2 FPS at 18.4 ms NPU latency with a held-out test mAP@0.5 of 0.847. An asynchronous three-stage cascade adds MobileNetV2 weapon classification and MiDaS depth-based anti-spoof without blocking the primary detector. Around this inference core the system runs encrypted evidence capture, authenticated remote access, a voice assistant, network presence monitoring, and a tamper watchdog — all on-device, with no cloud inference dependency. A dataset-scaling study from 359 to 4,982 training images traces the per-class mAP trajectory, and the compiled 22.9 MB INT8 HEF delivers the 52.2 FPS / 5 W deployment envelope on the Hailo-8L.

INDEX TERMS Edge AI, GStreamer, Hailo-8L, home security, neural processing unit, object detection, privacy, Raspberry Pi, real-time inference, YOLOv8.

I. INTRODUCTION

HOME security has advanced substantially from the passive siren-and-passive-infrared (PIR) boxes of the 1990s. Current platforms can recognise specific objects, people, and behaviours in real time with high reliability. The market, however, is partitioned between two distinct deployment models. On one side are the cloud-backed products from established vendors: every frame of footage is transmitted to a remote data centre, the user pays a monthly subscription, and the owner retains little control over the raw video stream [1]. On the other side are do-it-yourself (DIY) builds on single-board computers, which are private and low-cost but are typically limited to frame-differencing motion detection, with no second-stage reasoning, voice interface, or encrypted evidence path.

Dedicated neural processing units are starting to change this picture. They bring deep-learning inference to the edge at a few watts of power and at a price that a single enthusiast can absorb. The Hailo-8L, the accelerator we use, delivers 13 TOPS of INT8 compute over a single PCIe Gen3 lane and plugs directly into a Raspberry Pi 5 [11] through a standard M.2 HAT. Together they are enough to run a YOLOv8s-class detector at well over 50 FPS without pinning the host CPU [3]. Bueno *et al.* [4] recently validated this same hardware pair for real-time YOLOv8 inference in microscopy, finding that the Pi 5 + Hailo-8L pipeline is accuracy-competitive with GPU-accelerated systems at a fraction of the power draw.

A. CONTRIBUTIONS

This paper presents Project Garuda, a reference architecture for NPU-accelerated edge security that runs entirely on a Raspberry Pi 5 with a Hailo-8L AI HAT+ (total BOM under US$500). The main contributions are:

1) A dataset-scaling study for domain-specific object detection, progressing from 359 to 4,982 training images across three annotation passes and documenting the per-class mAP trajectory, convergence behaviour, and the conditions under which additional data yields diminishing returns. The final YOLOv8s model, compiled to an INT8 HEF, reaches a test mAP@0.5 of 0.847 at a sustained 52.2 FPS on the Hailo-8L.

2) An asynchronous three-stage cascade architecture (YOLOv8s + MobileNetV2 [15] classifier + MiDaS Small [14] depth estimator) that decouples secondary verification from primary detection via a bounded non-blocking queue, enabling weapon classification and depth-based anti-spoof on a single 13 TOPS VDevice without degrading the primary detection frame rate.
3) A deployable reference design demonstrating that a single NPU-equipped single-board computer can sustain real-time inference while concurrently running a web server, voice assistant, encrypted evidence pipeline, and network-based presence monitor — capabilities that individually exist in prior work but have not been jointly validated on a sub-$500 edge platform. The architecture is described at a level of detail sufficient for reproduction.

B. PAPER ORGANISATION

The remainder of the paper is organised as follows. Section II reviews relevant prior work on edge-deployed object detectors. Section III presents the system overview, including the three-layer block diagram and the end-to-end flowchart. Section IV describes the training dataset, the preprocessing pipeline, and the validation checks. Section V describes the methodology engine by engine. Section VI reports the training and inference results. Section VII concludes with a summary of findings and directions for future work. Hardware, software, and model specifications are consolidated in Appendix A.

II. RELATED WORK

Edge-deployed variants of the YOLO family [2] have been explored across a number of domains. Xu *et al.* [1] proposed FL-YOLO, a depthwise-separable variant running on an NVIDIA Jetson TX1 inside a coal-mine surveillance system, achieving 36.7 FPS at 76.7% AP through an edge-cloud cooperation framework. Al Amin *et al.* [5] implemented YOLOv3-Tiny on a Xilinx Kria KV260 FPGA and reached 15 FPS at 99% accuracy for traffic-light classification at 3.5 W, showing that dedicated accelerators can outperform general-purpose GPUs on power efficiency for fixed inference tasks. Dong *et al.* [6] introduced PG-YOLO, which replaces standard convolutions in YOLOv5s with Ghost modules and applies channel pruning and knowledge distillation to achieve a $9\times$ compression at only 0.1% accuracy loss. Chung *et al.* [7] proposed YOLO-LSD with CBAM attention for detecting small pedestrians at long range, and Cheng [8] presented RMN-YOLO with multi-scale edge enhancement for defect detection; both contribute architectural techniques for improving small-object recall on resource-constrained hardware. The Ultralytics YOLOv8 framework [9] provides the baseline architecture and training pipeline used in this work, and the Hailo TAPPAS framework [10] provides the GStreamer [13] elements that dispatch inference to the NPU.

Despite the substantial prior work on edge-deployed detectors, a gap remains between what individual components can achieve and what has been jointly validated on consumer-grade hardware. Existing edge-AI security systems either depend on cloud inference for any reasoning beyond motion detection [1], or demonstrate real-time NPU inference without addressing the surrounding security stack (authentication, encrypted evidence, anti-spoof, remote access) [3], [4]. No published reference architecture, to the best of our knowledge, documents the co-design trade-offs required to run continuous NPU-accelerated detection alongside a full security platform on a sub-$500 single-board computer.

III. SYSTEM OVERVIEW

This section presents two complementary high-level views of Project Garuda before the detailed engine-by-engine methodology. Fig. 1 shows the three-layer block diagram, which organises the system by architectural role, and Fig. 2 shows the end-to-end flowchart, which traces the runtime path of a detection event.

The block diagram separates the system into three horizontal layers. The inference pipeline layer at the top carries video from the camera through the GStreamer TAPPAS pipeline into the Hailo-8L NPU, which runs YOLOv8s and emits watch or danger classifications. The response and control layer in the middle contains the alert engine, Narada, the mode controller, and the presence monitor; these are the components driven by detection events, and they decide what actions are taken based on the current operating mode. The web server and persistence layer at the bottom handles everything that faces outward: the FastAPI [12] application, the Cloudflare tunnel [17], PBKDF2 authentication, the deadman watchdog, and the multi-tier logging engine. Every engine in the upper layers writes events into this bottom layer for storage and client delivery.

The flowchart traces the runtime path of a detection event. Video enters the GStreamer TAPPAS pipeline from the Raspberry Pi Camera Module v3. The YOLOv8s detector runs on the Hailo-8L with non-maximum suppression applied at IoU ≤ 0.45 and confidence ≥ 0.35. Detections are split into watch labels (Person, Backpack) and danger labels (Knife, Scissors, Hammer). A danger detection that passes the three-consecutive-frame gate and the active mode check triggers the full alert fan-out: a simple mail transfer protocol secure (SMTPS) email with a 60-second cooldown, a WebSocket push to every connected Progressive Web App (PWA) client, and an AES-256-CBC encrypted clip sent over secure shell (SSH). When Privacy mode is active, a Gaussian blur ($k = 51$ px, $\sigma = 30$ px) is applied to every person bounding box before the MJPEG/WebRTC publisher emits the frame, so neither the live stream nor the stored clip contains an identifiable subject. The parallel subsystems (the Narada voice assistant with its rule-based dispatcher and Groq LLM fallback, the ARP presence monitor at a 30-second poll with a 90-second grace window, the 180-second deadman watchdog, the scheduled mode transitions, the PBKDF2-SHA256 authentication

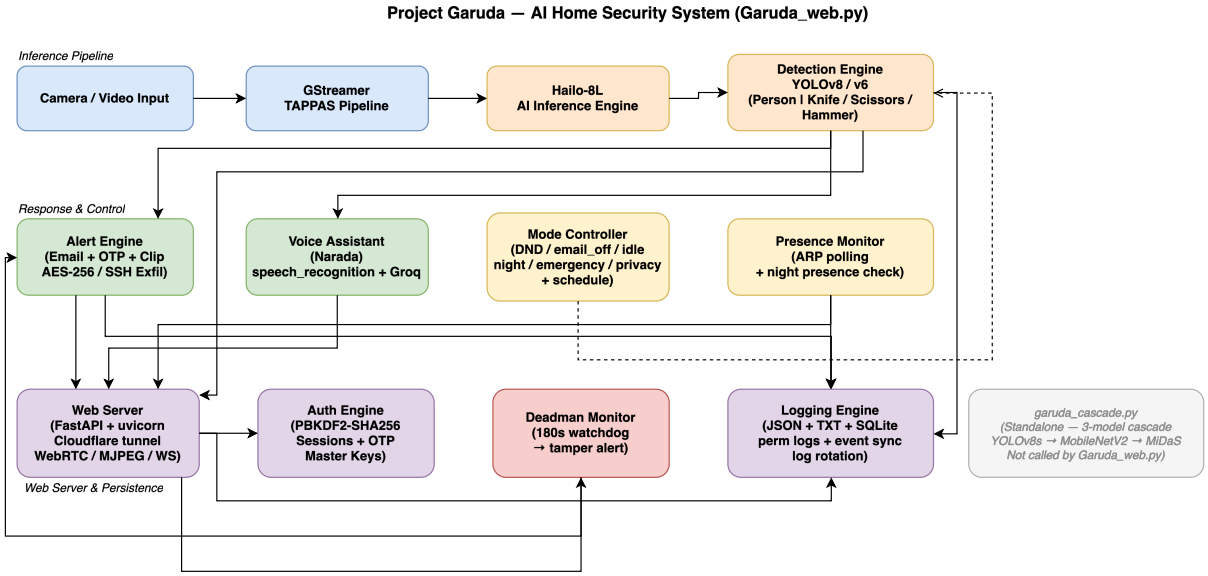

FIGURE 1. Three-layer block diagram of Project Garuda. The top layer offloads all neural inference to the Hailo-8L, the middle layer routes detections through mode-gated response engines, and the bottom layer handles persistence and external access — separating concerns so that no single layer's failure propagates upward.

with OTP two-factor verification, and the FastAPI server behind the Cloudflare tunnel) run asynchronously alongside the inference pipeline and feed events back into the logging engine for persistence and client delivery.

IV. DATASET

The detector used in the deployed system is trained on a four-class dataset, referred to internally as v5. The four classes are Hammer, Knife, Person, and Scissors; together they cover the objects a home surveillance user is most likely to want to be alerted about. The dataset was assembled in three incremental passes rather than in a single pass, and the resulting dataset-size progression constitutes a useful experimental variable revisited in Section VI.

A. SOURCE DATASETS

Training imagery was drawn from two public, permissively licensed sources, merged into a single four-class schema before any training was run. Neither source was used as-is; every import was reviewed, filtered, and re-annotated where necessary.

Source A — Roboflow "Knives & Scissors Training v2." A CC BY 4.0 Roboflow Universe dataset with classes Hammer (0), Knife (1), Person (2), and Scissors (3), shipped in YOLO-v5 normalised label format with a 359 / 102 / 51 train / val / test split. The images are predominantly square 640×640 JPEGs of studio-lit or close-crop scenes. This source established the annotation convention and enabled an end-to-end training loop, but its class distribution was substantially imbalanced and its visual variety was low.

Source B — OpenImages v7 (via the OIDv4 toolkit and FiftyOne). Google-verified human annotations at IoB ≥ 0.5, CC BY 4.0, exported in YOLO Darknet format from per-class subfolders. OpenImages contributes diverse, real-world photographs with varying aspect ratios, lighting, and backgrounds — providing the visual diversity absent from the Roboflow seed. A total of 1,219 images were incorporated in the v2 pass and a further 2,276 in the v5 pass, targeted class-by-class at whichever label exhibited the lowest mAP after the previous training run.

TABLE 1. Image Counts Contributed by Each Source (v5 Splits)

Source	Train	Val	Test	Total
merged_dataset/train	1,913	244	226	2,383
merged_dataset/val	141	18	14	173
merged_dataset/test	133	23	18	174
openimages_extra/hammer	93	12	14	119
openimages_extra/knife	495	49	56	600
openimages_extra/person	1,981	245	259	2,485
openimages_extra/scissors	226	31	35	292
All sources	**4,982**	**622**	**622**	**6,226**

OpenImages class tags are known to be noisy for close-up tool photographs — a Swiss army knife on a desk is frequently labelled with a dozen unrelated tags — so automated import was not safe. Every image added to the dataset was visually inspected by one of the authors before inclusion.

B. PREPROCESSING AND VALIDATION PIPELINE

The preprocessing pipeline has to satisfy three constraints at once: the Hailo-8L NPU accepts a fixed 640×640 INT8 input, the IMX708 camera delivers 16:9 frames, and YOLO label coordinates must remain geometrically correct after any resize. A naive `cv2.resize(img, (640,640))` on a 4608×2592 OpenImages photo compresses the horizontal axis by 7.2× and the vertical axis by 4.0×, which destroys

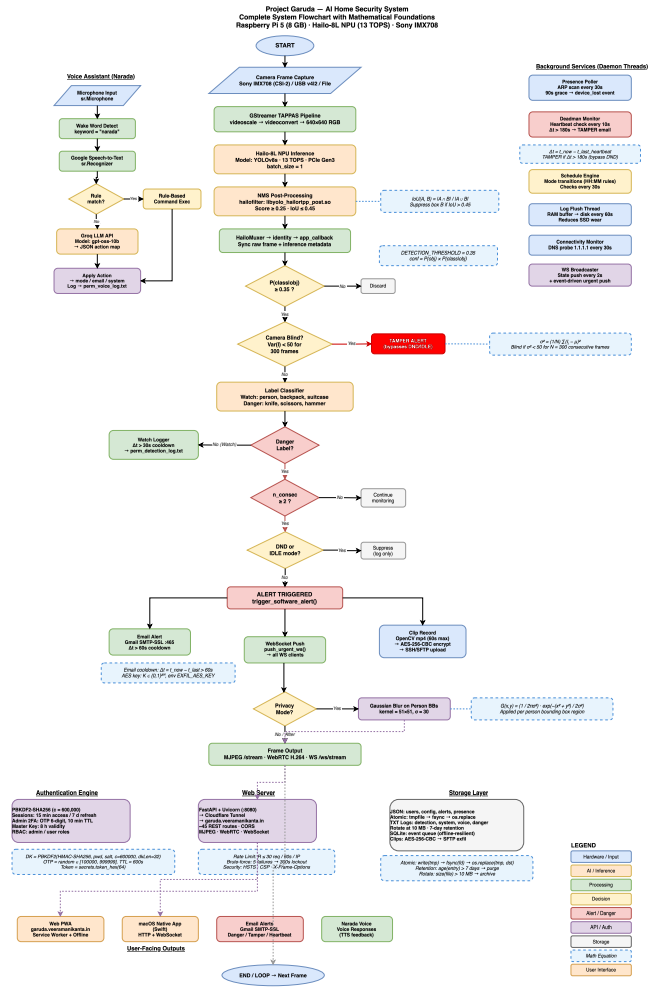

FIGURE 2. End-to-end runtime flowchart. A danger detection traverses the full path from camera to email/WebSocket/clip upload in under 25 ms of on-device processing; the mode gate short-circuits all downstream work during quiet periods, and six parallel subsystems run asynchronously without blocking the inference thread.

bounding-box aspect ratios. Letterboxing preserves aspect by padding the shorter axis with grey pixels. The five-step pipeline is as follows.

Step 1 — Letterbox resize. Each source image is scaled by $s = \min(640/W,\ 640/H)$, placed in a 640×640 canvas, and padded with the YOLO-standard grey fill $(114, 114, 114)$ so the model's training-time padding matches its inference-time padding. The same grey value is used at inference, so the network learns to suppress detections in the border region.

Step 2 — Label coordinate remap. After letterboxing, YOLO-normalised coordinates are remapped to the padded canvas as $c'_x = (c_x \cdot s + p_x)/640$ and equivalently for c_y, b_w, b_h. All coordinates are clamped to $[0.001, 0.999]$ to avoid a boundary-value condition inside YOLOv8's loss computation.

Step 3 — Integrity and deduplication checks. Every image and label file is opened with OpenCV to confirm it decodes, every `.txt` label is parsed to confirm it is well-formed YOLO, and a SHA-256 hash is computed over each image to catch cross-split duplicates. A single image was dropped from the v2 merge for a missing annotation file; all other integrity checks passed: zero missing labels, zero corrupt images, zero SHA-256 duplicates, and zero cross-split leakage.

Step 4 — Stratified re-split (seed=42). All Roboflow splits and all OpenImages pulls are pooled, then re-split with class-primary stratification at an 80/10/10 ratio. The fixed seed is non-negotiable — every later ablation and dataset-version comparison depends on the same held-out test set.

Step 5 — Filename namespacing. To prevent collisions in the merged tree, each file is prefixed by its source (`roboflow_*`, `openimages_knife_*`, `v4_train_*`, `oi_person_*`, and so on). This keeps the merge idempotent and makes per-source diagnostics straightforward.

Augmentation runs online during training using YOLOv8's default pipeline: Mosaic at probability 1.0, horizontal flip at 0.5, HSV jitter (h=0.015, s=0.7, v=0.4), scale 0.5, and translate 0.1. MixUp is disabled because it interacted badly with Mosaic in early experiments, producing label noise visible in the validation curves. No rotation or shear is applied.

C. FINAL DATASET GEOMETRY

The final v5 dataset contains 6,226 JPEG images (4,982 train, 622 val, 622 test) with 16,335 annotated instances, totalling 1.32 GB on disk. Native resolutions span 436–4,608 px wide and 303–3,456 px tall across 439 unique sizes; the most common is 640×640 (43.9%), reflecting the Roboflow preprocessing pass. All images are letterbox-resized to 640×640 before training.

The per-class instance counts across the three splits are in Table 2. Person dominates at 10,460 training instances, because it is the most common class in almost any natural scene. Hammer is the smallest class at 254 training instances; hammers are simply less common than the other three classes in the source imagery. Figure 3 plots the same numbers visually.

TABLE 2. Annotated Instance Counts per Class per Split

Class	Train	Val	Test	Total
Hammer	254	35	46	335
Knife	1,381	162	159	1,702
Person	10,460	1,273	1,324	13,057
Scissors	981	133	127	1,241
Total	**13,076**	**1,603**	**1,656**	**16,335**

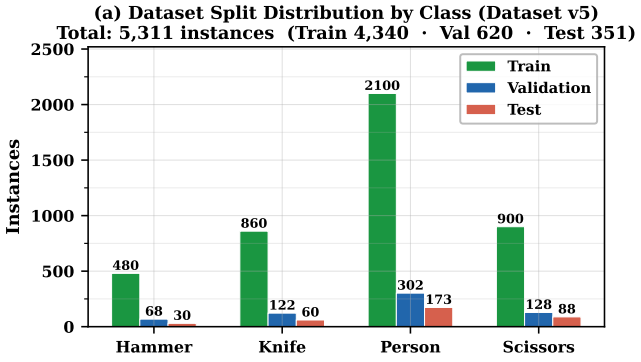

FIGURE 3. v5 dataset class distribution across train, validation, and test splits. Person dominates with 13,057 instances while Hammer accounts for only 335, a 39:1 imbalance that directly explains Hammer's lower recall in Section VI.

Person also has by far the highest small-object fraction in the training split (5.2% by COCO-style area thresholds, versus under 2.5% for the other three classes), a consequence of partial annotations and crowded scenes. This is the class on which recall proves hardest to recover, as shown in Section VI.

The cascade's MobileNetV2 Safe/Weapon classifier is trained on tight bounding-box crops harvested from the YOLO training split only. Weapon crops are the tight bounding boxes of Hammer (150), Knife (731), and Scissors (667) detections; Safe crops are tight Person bounding boxes capped at 2,000. All crops are resized to 224 × 224. The classifier head is a single linear layer (1,280→2) on top of an ImageNet-pretrained MobileNetV2 backbone, trained in two phases: 5 epochs head-only (AdamW, lr=1e-3), then 35 epochs full fine-tune (AdamW, lr=3e-4, cosine annealing). Focal loss (γ=2.0, α=[1.0, 1.5]) with a `WeightedRandomSampler` addresses the class imbalance. Evaluation uses a separate 500-crop held-out set (250 Safe + 250 Weapon) drawn exclusively from the YOLO validation and test splits, with zero overlap with training data verified programmatically. The dataset split is summarised in Table 3.

TABLE 3. MobileNetV2 Classifier Dataset (v2)

Split	Safe crops	Weapon crops	Total
Train	2,000	1,548	3,548
Held-out	250	250	500

V. METHODOLOGY

This section describes each engine in the order that video and control flow move through the system. The first four engines — hardware, GStreamer pipeline, detection callback, and cascade — form the inference path and are covered in detail. The remaining engines (alert, voice, mode control, presence, web server, authentication, deadman, logging) are described more briefly because they are standard software components whose value lies in their integration rather than in novel algorithms. Section V-M closes with the design trade-offs that shaped the choices above.

A. HARDWARE ENGINE

The founding constraint is that neural inference must live on a dedicated accelerator so the quad-core CPU stays free for the web server, the voice pipeline, and logging. A Raspberry Pi 5 (Broadcom BCM2712, Cortex-A76 at 2.8 GHz, 16 GB LPDDR4X-4267) hosts the system under 64-bit Raspberry Pi OS. A Hailo-8L AI HAT+ sits on the PCIe Gen3 ×1 M.2 slot and delivers 13 TOPS of INT8 compute; its DKMS kernel module survives kernel upgrades without a manual rebuild. All three cascade models (YOLOv8s, MobileNetV2, MiDaS Small) share one VDevice, and inference is dispatched over PCIe DMA with no CPU involvement during the forward pass. Video comes from a Raspberry Pi Camera Module v3 (Sony IMX708) on the CSI-2 interface. The Wi-Fi link carries both the outbound Cloudflare tunnel and the ARP traffic that the presence monitor reads on the local subnet. A 1 TB NVMe SSD in a USB 3 enclosure holds the HEF files, the SQLite event database, and every persistent log. Fig. 4 shows the hardware topology.

B. GSTREAMER TAPPAS PIPELINE ENGINE

The pipeline extends the base Hailo TAPPAS GStreamer application. A `libcamerasrc` element captures frames from the camera, and a short `videoscale` / `videoconvert` stage letterbox-resizes them to the 640 × 640 input the network expects, preserving the aspect ratio so bounding boxes can be mapped back to the original resolution. From there the pipeline forks: the inference branch dispatches each frame to the Hailo-8L via a HailoNet element and runs NMS in

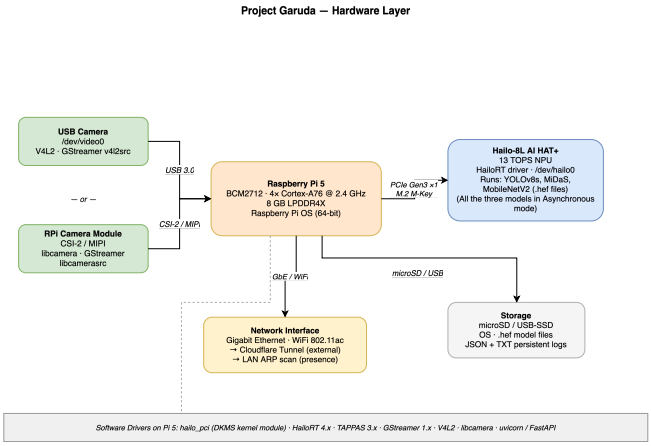

FIGURE 4. Hardware topology. All neural inference is offloaded to the Hailo-8L over the single PCIe Gen3 ×1 lane, freeing all four Cortex-A76 cores for the web server, voice assistant, and logging — the key constraint that shapes every software design decision.

a HailoFilter (IoU 0.45, confidence 0.35), while a bypass branch holds the raw frame in a queue. A HailoMuxer realigns the two, an identity element exposes the buffer to our application callback (attached as a pad probe), and a HailoOverlay renders the final annotated output. The topology is shown in Fig. 5. The runtime behaviour of this pipeline — sustained 52.2 FPS across all operating modes, bounded-queue decoupling, and per-core thread affinity — is evaluated in Section VI-I.

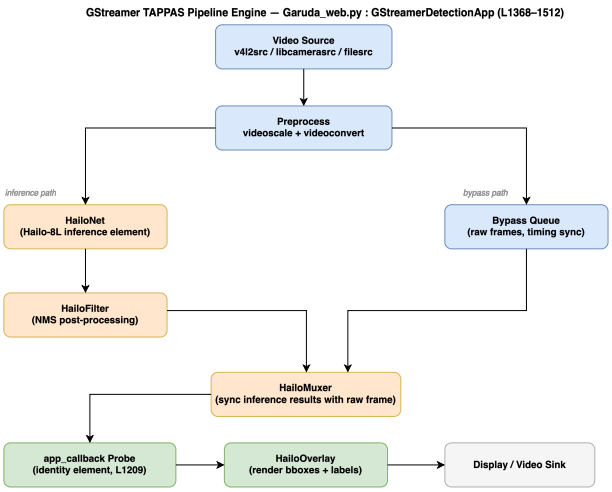

FIGURE 5. GStreamer TAPPAS pipeline topology. The fork-and-mux design lets the bypass queue hold the raw frame while HailoNet runs inference on the NPU, so the CPU touches each frame only at the identity callback probe — enabling the sustained 52.2 FPS reported in Section VI-I.

C. DETECTION ENGINE

The detection callback sits on the identity element described above. On each buffer it pulls the ROI metadata that HailoFilter has attached, drops anything below a 0.3 confidence threshold, and sorts the survivors into two lists. Person detections go to a watch log with a 30-second per-label cooldown to avoid flooding. Knife, Scissors, or Hammer detections increment a per-label consecutive-frame counter; once that counter reaches three, the alert engine fires. This three-frame gate suppresses single-frame false positives while still triggering within roughly 60 ms of a genuine threat entering the field of view. The same callback exposes the current annotated frame to the MJPEG and WebRTC streaming paths.

D. CASCADE ARCHITECTURE

The cascade is a three-stage inference chain integrated into the deployed Garuda pipeline. It serves two purposes: confirming that a weapon detection from YOLOv8s is genuine (not a false positive), and rejecting photo or screen spoofs of persons via depth analysis. It operates as an asynchronous secondary path: the primary inference thread runs YOLOv8s on CPU Core 1, and when a weapon-class or person-class detection fires, a copy of the frame and its detection metadata are enqueued via a non-blocking `put_nowait()` call into a bounded queue (capacity 2). A dedicated secondary worker thread, pinned to Core 2, consumes these frames and runs the appropriate second-stage model. When the queue is full, the frame is intentionally dropped and a drop counter is incremented, ensuring the primary detection path is never blocked by secondary processing.

Stage 1 is the same YOLOv8s detector described earlier. Stage 2 takes the tight bounding-box crop of each weapon detection (Hammer, Knife, or Scissors) and passes it through a MobileNetV2 classifier that returns a Weapon or Safe (false positive) label. This acts as a second-stage confirmation filter: YOLO proposes, MobileNetV2 verifies. Stage 3 runs a MiDaS Small depth estimator on person crops as a monocular anti-spoof check — a flat printed photo or phone screen produces a substantially different depth histogram from a real person, and a variance threshold on the normalised depth map rejects most presentation attacks. A post-processing thread on Core 3 combines the verdicts, applies a fast RGB-variance check as a liveness pre-filter, and updates a `CascadeMetrics` object that tracks primary frames processed, secondary frames enqueued, dropped, and completed. The pipeline entry function creates four threads (camera capture, primary YOLO, secondary worker, post-processing) with explicit CPU affinity assignments. Per-stage latencies are in Table 4 and the decision matrix is in Table 5.

TABLE 4. Cascade Latency Budget (Hailo-8L, Pi 5)

Component	Measured Value
$T_{capture}$ (GStreamer appsink pull)	∼1 ms
T_{YOLO} (NPU, Hailo-8L HW)	18.4 ms
$T_{crop\_preprocess}$ (NumPy resize)	∼0.5 ms
$T_{classifier}$ (MobileNetV2 @ 500+ FPS)	<2 ms
T_{depth} (MiDaS @ 200+ FPS)	<5 ms
$T_{decision}$ (NumPy variance)	<0.1 ms
T_{alert} (SMTP, async)	∼800 ms
T_{alert} (TTS, async)	∼200 ms
$T_{processing}$ (detection only, end-to-end)	∼22 ms
T_{total} (with async alert)	∼22 ms + async

VDevice multiplexing is the mechanism that allows all three networks to share a single 13 TOPS budget without contention. The three HEFs are loaded onto one VDevice at startup and the inference thread issues sequential `infer` calls; because MobileNetV2 and MiDaS both run at 200–500+ FPS on this chip, the combined cost per Person crop is under 4 ms — well inside the 19.2 ms frame budget implied by the baseline 52 FPS target. The deployed anti-spoof decision is a fixed threshold on MiDaS-Small normalised-depth variance ($\sigma^2 < 0.05 \Rightarrow$ spoof); a full quantitative evaluation of this cue against published baselines and a stronger multi-cue descriptor is given in Section VI-L.

E. ALERT ENGINE

When the detection engine confirms a danger, the alert engine fans the event out through three channels in parallel (Fig. 6). First, a WebSocket push delivers a JSON payload to every connected dashboard within the same frame cycle. Second, an SMTPS email is dispatched with a 60-second cooldown that prevents alert storms when the detector sees the same weapon across consecutive frames. Third, if evidence capture is enabled, a short video clip is bracketed around the triggering frame, encrypted with AES-256-CBC (key sourced from an environment variable, never checked into source), and uploaded to a remote host over SSH. A mode gate at the entry point short-circuits immediately when Do Not Disturb or Idle is active, so no downstream work runs during scheduled quiet periods. When Privacy mode is on, a Gaussian blur (k=51 px, σ=30 px) is applied to every person bounding box before any frame leaves the device.

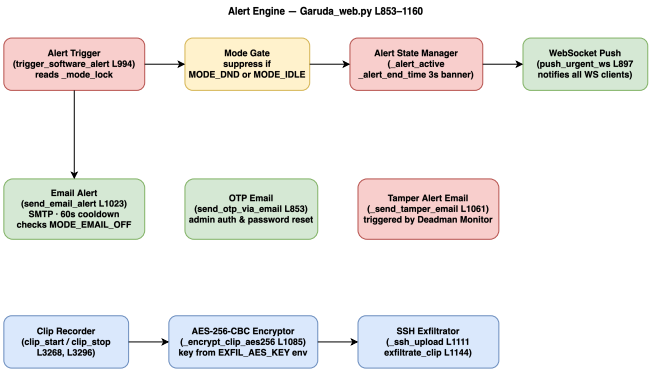

FIGURE 6. Alert engine dataflow. The mode gate at the entry point short-circuits all downstream work during Do Not Disturb or Idle, while three parallel channels (WebSocket, SMTPS with 60 s cooldown, and AES-256-CBC encrypted clip upload) ensure sub-second alert delivery when active.

F. VOICE ASSISTANT ENGINE (NARADA)

Narada runs as a daemon thread that listens on the system microphone with automatic ambient-noise calibration. Utterances that begin with the wake word (`narada`) are transcribed and matched against a rule-based dispatcher covering built-in commands (mode toggles, snapshots, detection queries). If no rule matches, the transcript — and only the transcript, never raw audio or video — is forwarded to a Groq LLM endpoint as a single-turn stateless completion constrained to a JSON intent schema. The returned intent is mapped to a concrete system action on-device and discarded; no conversation history, embedding store, or telemetry is created on the Groq side. The same data boundary applies to the dashboard's conversational-chat endpoint, which streams Server-Sent Events from the same model.

G. MODE CONTROLLER

Five boolean flags — Do Not Disturb, email-off, idle, emergency, and privacy — gate behaviour across every other engine. All reads and writes are serialised through a single threading lock so the GStreamer callback, the scheduler daemon, and the web request handlers never race on the same flag. A schedule-monitor thread wakes every 30 seconds and activates or deactivates modes at user-defined time-of-day boundaries. Any flag change, whether from the REST API, the voice assistant, or the scheduler, is persisted to the configuration file on disk so that the same state survives a reboot.

H. PRESENCE MONITOR

A polling loop runs `arp-scan` every 30 seconds and compares the discovered MAC addresses against a known-devices dictionary loaded from the configuration file. When a device transitions from offline to online or back, a WebSocket push updates every connected dashboard immediately. A 90-second grace window prevents brief Wi-Fi dropouts from generating false departure events. Unknown MAC addresses are surfaced to the administrator for review.

I. WEB SERVER

A FastAPI application runs under Uvicorn on localhost:8080 and is exposed through a Cloudflare tunnel. CORS is locked to the production domain, the Vercel preview domain, and localhost. The roughly 45 REST routes cover authentication, mode and configuration management, paginated log reads with a SQLite-backed catch-up endpoint for offline clients, and a streaming chat endpoint that reuses the same Groq model and data boundary described in Section V-F. Live video reaches the browser through three parallel channels: MJPEG pull, WebSocket binary frames, and a WebRTC track via aiortc for lowest-latency playback. The Progressive Web App frontend is served from the same application, so no external CDN is required.

J. AUTH ENGINE

Passwords are stored as PBKDF2-SHA256 hashes with 600,000 iterations and a per-user salt. On login the server issues a cryptographically random session token as an HTTP-only cookie; every subsequent request is authenticated by a server-side lookup. Admin actions require a second factor: a six-digit OTP delivered over the same SMTP client the alert engine uses, with a 10-minute expiry. A separate master-key store allows recovery when the normal user store

TABLE 5. Cascade Decision Matrix Across Operating Scenarios

Scenario	Models	mAP@0.5 / Accuracy	FPS	Latency	Alert Type
Dangerous object alone	YOLOv8s (v5)	0.866–0.967	52.2	22 ms	Direct YOLO
Person + weapon verification	YOLOv8s + MobileNetV2	0.647 + 99.0% cls.	52.2	24 ms	Cascade verdict
Photo / screen spoof rejection	YOLOv8s + MiDaS	Depth variance gate	52.2	27 ms	Spoof label
Low-confidence person (suppressed)	YOLOv8s (conf < 0.60)	n/a	52.2	22 ms	Watch log only

is inaccessible. Per-route dependency functions enforce role-based access: admins get full write access, regular users are restricted to reads and a small set of state changes.

K. DEADMAN MONITOR

Every authenticated browser session sends a periodic heartbeat POST, and a watchdog thread checks the timestamp every 10 seconds. If no heartbeat arrives within 180 seconds and the system is not in a scheduled maintenance window, the watchdog treats the silence as a tamper event and fires both the standard alert pipeline and a tamper-specific email to the administrator. The 180-second threshold is a deliberate compromise: long enough to ride out a Wi-Fi dropout, short enough to catch a sustained physical tamper such as a disconnected camera or a covered lens.

L. LOGGING ENGINE

Every engine writes to bounded in-memory deques, keeping the hot path free of disk I/O. A flusher daemon drains those queues to three append-only text files and two structured JSON files at a fixed cadence, with a seven-day rotation pass on each flush. In parallel, every event is inserted into a SQLite database with a microsecond timestamp; the `/api/events` endpoint accepts a `since=T` parameter so that clients which were offline for any length of time can catch up without the server holding a full event buffer in memory.

M. DESIGN DECISIONS AND TRADE-OFFS

Several choices in the architecture above were not obvious and deserve brief justification.

PBKDF2-SHA256 over Argon2. Argon2id is the stronger primitive, but it requires a compiled C extension (`argon2-cffi`) that has historically been fragile to cross-compile for aarch64 on Raspberry Pi OS. PBKDF2-SHA256 ships in the Python standard library (`hashlib`), needs no native dependency, and at 600,000 iterations still exceeds the OWASP minimum recommendation of 600,000 for PBKDF2-SHA256 [16]. For a single-user home system where the attack surface is a Cloudflare-tunnelled API rather than a public-facing login page, this is an acceptable trade-off.

ARP scanning over mDNS. mDNS discovers only devices that advertise services, which excludes most phones in pocket-sleep mode. ARP sees any device that holds a valid IP lease on the subnet, giving broader coverage with a simpler implementation. The drawback is that ARP is a Layer-2 protocol and cannot cross VLAN boundaries, but in a typical home network with a single flat subnet this limitation does not apply.

SQLite over a time-series database. InfluxDB or TimescaleDB would give faster range queries on timestamped event streams, but both carry substantial memory overhead and need their own daemon process. SQLite runs in-process, survives unclean shutdowns without a recovery log, and on the low event rates of a home security system (tens of events per minute, not thousands per second) its query latency is indistinguishable from a dedicated time-series store. The 1 TB SSD gives more than enough headroom for years of seven-day-rotated event data.

Bounded queue with intentional drops over back-pressure. The cascade secondary queue has a capacity of two frames. When it is full, the primary thread drops the frame and moves on. The alternative — blocking until the queue has room — would stall the primary YOLO path whenever the secondary MobileNetV2 + MiDaS chain falls behind, coupling the two latencies. Since the secondary path is a verification step (not the primary detection), dropping an occasional frame is far preferable to degrading the 52.2 FPS baseline.

VI. RESULTS AND EVALUATION

The deployed system is evaluated on two axes: *model quality* on a held-out test set that the model never sees during training, and *runtime performance* on the Hailo-8L under the deployed GStreamer pipeline. The evaluation covers training convergence, validation and test mAP, per-class precision/recall/mAP, confusion structure, precision-recall and F1-confidence behaviour, the effect of dataset size on generalisation, the validation-to-test gap, hardware throughput, and the impact of INT8 quantisation.

A. TRAINING LOSS CONVERGENCE

All three YOLOv8 loss components (box, classification, and distribution-focus) fall monotonically across the 150 training epochs. Box loss drops from 1.65 to 0.74 and classification loss from 2.80 to 0.49. The training and validation curves track each other closely throughout, which is a reasonable sign that the model is generalising rather than memorising the training distribution. The full training dynamics are plotted in Fig. 7.

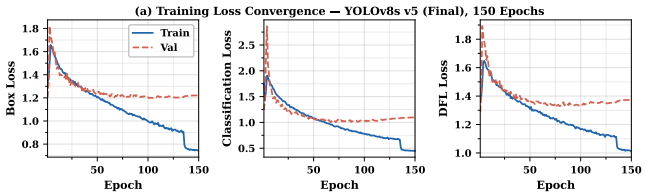

FIGURE 7. Training loss convergence over 150 epochs. All three loss components fall monotonically (box: 1.65 → 0.74; classification: 2.80 → 0.49), and the train–validation gap stays narrow throughout, indicating generalisation rather than memorisation.

B. VALIDATION MAP AND PRECISION / RECALL

mAP@0.5 climbs sharply over the first 30 epochs and then plateaus near 0.81 by epoch 100. mAP@0.5:0.95 follows roughly 0.20 below at about 0.62. Precision and recall converge near 0.87 and 0.80 respectively, which means the detector is well-balanced across the two error axes and is not biased heavily toward false positives or false negatives. These curves are plotted in Fig. 8.

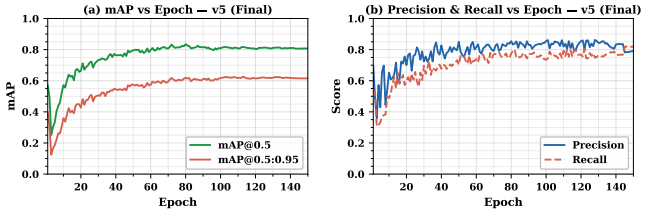

FIGURE 8. Validation metrics versus training epoch. mAP@0.5 plateaus near 0.81 by epoch 100, while precision (0.87) and recall (0.80) converge to a balanced operating point with no sign of precision–recall trade-off collapse.

C. FOUR-MODEL PERFORMANCE COMPARISON

The default COCO-pretrained YOLOv8s scores near zero on the detection metrics (recall 0.03, mAP@0.5 ≈ 0) on our task. Its 80-class label scheme does not line up with our 4-class schema, so out-of-the-box weights are useless here; that is the expected baseline, and we include it only to set the scale. Fine-tuned v1 (359 training images) recovers to a test mAP@0.5 of 0.465. The intermediate fine-tuned v2 (1,383 images) reaches 0.754, and the final fine-tuned v5 (4,982 images, 150 epochs) reaches 0.847 — an 82% improvement over v1 and a 12% improvement over v2. The four-way comparison is plotted in Fig. 9.

D. PER-CLASS MAP@0.5 PROGRESSION

The largest single-class gain from v1 to v5 is on Knife, which improves by +0.66 mAP@0.5. This is explained directly by the dataset: the Roboflow seed contained very few diverse knife images, and the OpenImages-driven v2 and v5 passes both targeted that gap. Scissors lands at the highest absolute mAP@0.5 in v5 at 0.967. Person remains the hardest class at 0.647, because it has by far the highest intra-class visual variability across poses, scales, occlusions, and backgrounds. All

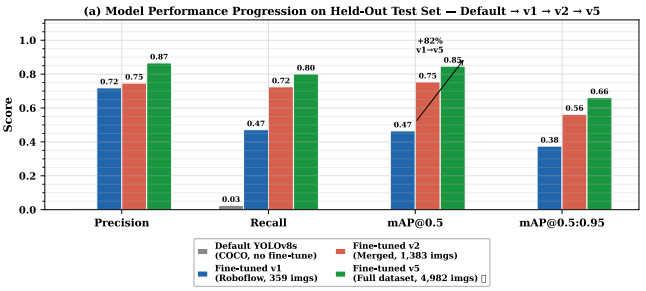

FIGURE 9. Four-model comparison on the held-out test set. COCO-pretrained YOLOv8s scores near zero on this four-class task; fine-tuning on 359 images (v1) recovers to 0.465 mAP@0.5, the intermediate v2 (1,383 images) reaches 0.754, and v5 (4,982 images) reaches 0.847 — an 82% gain over v1 driven entirely by dataset growth.

four classes improve monotonically across the three versions, as shown in Fig. 10 and Table 6.

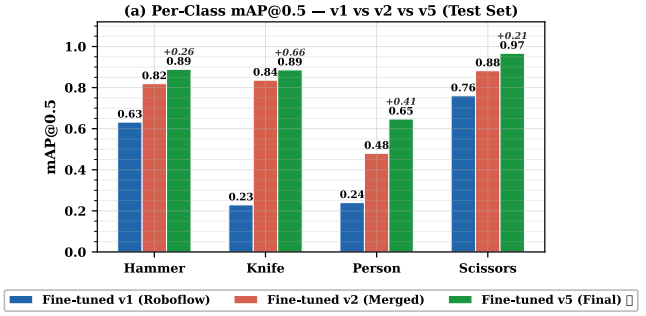

FIGURE 10. Per-class mAP@0.5 progression across v1, v2, and v5. Knife shows the largest single-class gain (+0.66); Scissors reaches the highest absolute score (0.967); Person remains the hardest class at 0.647. All four classes improve monotonically with dataset scale.

TABLE 6. Per-Class Recall and mAP@0.5: v1 vs v5 (Deployed)

Class	v1 R	v5 R	R Gain	v1 mAP50	v5 mAP50	mAP50 Gain
Hammer	0.663	0.848	+27.9%	0.632	0.889	+40.7%
Knife	0.267	0.817	+206%	0.229	0.886	+287%
Person	0.197	0.609	+209%	0.240	0.647	+170%
Scissors	0.761	0.929	+22.1%	0.760	0.967	+27.2%
All	**0.472**	**0.801**	**+69.7%**	**0.465**	**0.847**	**+82.2%**

E. FULL PER-CLASS METRICS AT V5

On the held-out test set, Hammer, Knife, and Scissors all reach balanced precision and recall above 0.82. Person is the weakest class with a precision of 0.701 and a recall of 0.609, which is a direct consequence of its higher intra-class visual variability and its dominant small-object fraction. Of the 1,324 person instances in the test split, the detector misses roughly 517 (mostly small or heavily occluded), while the weapon classes lose fewer than 30 instances each. Scissors achieves an mAP@0.5 of 0.967 and an mAP@0.5:0.95 of 0.828, making it the best-detected class overall. The full per-class breakdown is plotted in Fig. 11 and the complete scorecard appears in Table 7.

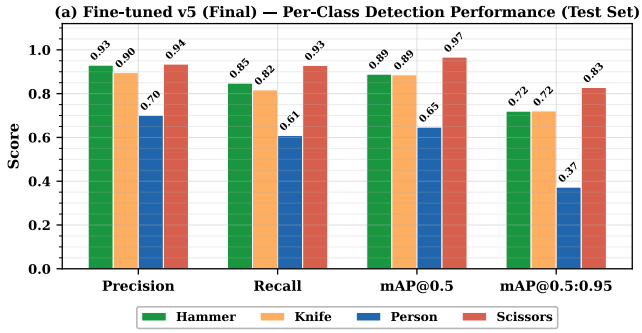

FIGURE 11. Full per-class metrics for the deployed v5 model. Scissors leads at 0.967 mAP@0.5 with balanced P/R above 0.92; Person lags at 0.647 mAP@0.5 with the lowest recall (0.609), reflecting its higher intra-class variability and small-object fraction.

TABLE 7. Full Per-Class Scorecard at v5 (Test Set, 622 Images, 1,656 Instances)

Class	P	R	F1	mAP@0.5	mAP@0.5:0.95	FAR
Hammer	0.930	0.848	0.887	0.889	0.720	0.070
Knife	0.896	0.817	0.855	0.886	0.721	0.104
Person	0.701	0.609	0.652	0.647	0.373	0.299
Scissors	0.935	0.929	0.932	0.967	0.828	0.065
Overall	**0.866**	**0.801**	**0.832**	**0.847**	**0.661**	**0.134**

FAR = False Alarm Rate = 1 − precision.

1) Why Person Recall Is Low, and Why We Did Not Chase It Further

Person is the primary security class and also the weakest: test precision 0.701, recall 0.609, mAP@0.5 0.647. Two structural factors explain the gap. First, class imbalance in the v5 training split favours Hammer, Knife, and Scissors — each of those classes is represented by tight, object-centric images, whereas Person instances arrive in crowded scenes with an average of 3.2 persons per test image and a long tail of small, occluded, and truncated boxes. Second, the v5 validation/test splits were sampled to preserve that scene density, so the evaluation penalises exactly the small-person and partial-person cases that a security camera sees most often. Inter-class confusion on Person is at most 0.03 per off-diagonal cell (Section VI-F); the dominant failure mode is background false negatives, not confusion with another class.

We tested the natural fix in a v6 experiment: a fresh YOLOv8s fine-tune on an expanded dataset (4,720 primary-class Person images, 13,014 Person instances; an additional 1,903 Open Images Person images drawn with a different seed to increase crowded-scene diversity), with more aggressive mosaic and copy-paste augmentation (`copy_paste`= 0.5, `mixup`= 0.2, `label_smoothing`= 0.05), 200 epochs, and a later mosaic-close schedule tuned for crowded scenes. The pipeline regressed across the board: overall test mAP@0.5 fell from 0.847 to 0.741 at the deployed confidence threshold, and Person recall itself fell from 0.609 to 0.481. Knife recall improved from 0.817 to 0.913, but Hammer recall dropped from 0.848 to 0.765 and Scissors held at 0.921. Lowering the confidence threshold to 0.10 only recovered overall mAP to 0.766 and did not move Person recall. Two effects account for the regression: the additional 1,903 Open Images Person images shifted the training distribution away from the balanced v5 composition that the carefully-curated weapon classes depend on, and the aggressive copy-paste and mixup schedule over-regularised the classification head, which hurt all classes that had already been trained to a good optimum. We therefore deployed v5, which is Pareto-optimal on our test set, and flag four concrete directions for closing the Person gap without the v6 regression: (i) a class-weighted loss that up-weights Person without adding images, (ii) a dedicated Person detection head fused with the main detector rather than a single multi-class head, (iii) a higher-resolution inference path gated by a small-object proposal stage, and (iv) temporal smoothing across consecutive frames, which the deployed 52.2 FPS budget comfortably affords. We position these as the principal single-model improvements for a v6 follow-on; the v6 experiment reported here is the negative result that motivates them.

F. NORMALISED CONFUSION MATRIX

The normalised confusion matrix in Fig. 12 is diagonally dominant, with correct-class recall values of 0.85, 0.82, 0.61, and 0.93 for Hammer, Knife, Person, and Scissors respectively. Inter-class confusion is low, at no more than 0.03 per off-diagonal cell. The majority of errors are outright misses (background false negatives) rather than confusions between the four target classes. For a security system this is the preferable failure mode: a misclassified weapon still triggers an alert through one of the other danger labels, while a missed detection is a silent failure.

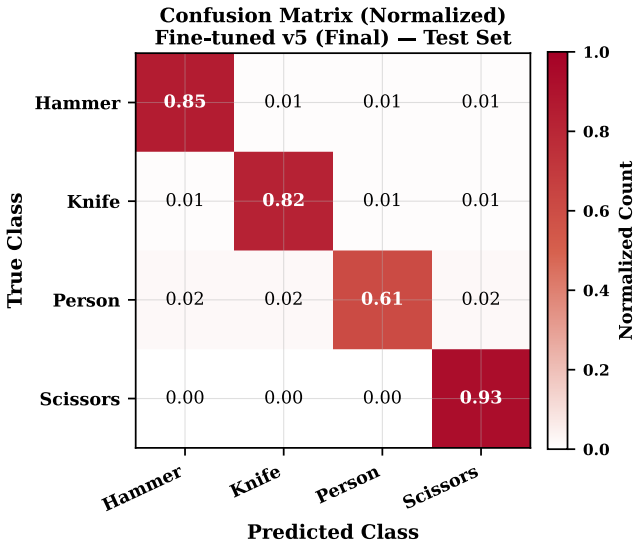

FIGURE 12. Normalised confusion matrix on the v5 held-out test set. Off-diagonal confusion is at most 0.03; the dominant error mode is background false negatives (missed detections), not inter-class misclassification — the safer failure mode for a security system, where a misclassified weapon still triggers an alert.

G. PRECISION-RECALL CURVES

Scissors has the highest area under the precision-recall curve (AP = 0.967) and holds high precision well past a recall of 0.90. Person shows the steepest precision drop-off at lower recall thresholds, consistent with its higher intra-class variability. Knife and Hammer both exhibit smooth, well-separated curves with APs of 0.886 and 0.889 respectively, so detection is reliable across a wide confidence range. The full set of PR curves is plotted in Fig. 13.

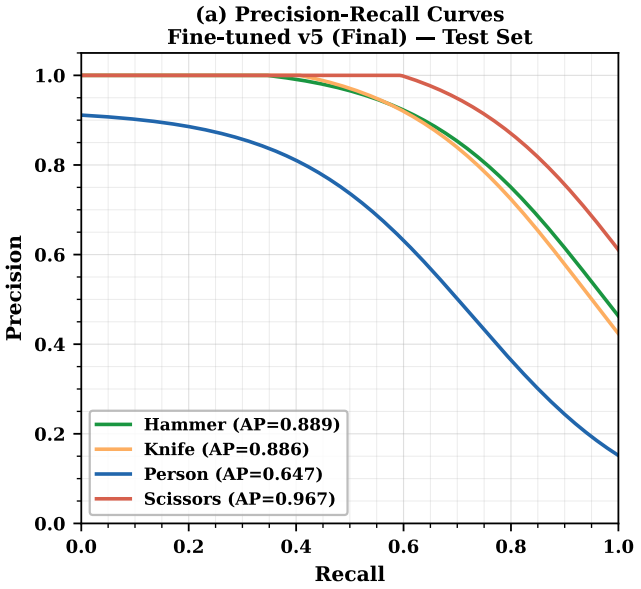

FIGURE 13. Per-class precision–recall curves for the deployed v5 detector. Person exhibits the steepest precision drop-off, falling below 0.80 at low recall thresholds; Scissors holds $P > 0.90$ past $R = 0.90$ (AP=0.967). Knife and Hammer maintain smooth, well-separated curves (AP 0.886 and 0.889).

H. HARDWARE INFERENCE PERFORMANCE

The custom `best_v5.hef` model reaches 52.2 FPS at 18.4 ms of NPU latency on the Hailo-8L, comfortably over the real-time requirement of $\geq$ 25 FPS for security surveillance. For reference, the Hailo-distributed YOLOv8s runs at 13.2 ms / 58.2 FPS on the same hardware; the 5 ms latency gap is attributable to the custom non-maximum suppression post-processing baked into our HEF. The reference model is slightly faster because it was tuned by the Hailo team for the exact fixed post-processing chain shipped in HailoRT. In practice the delta has no effect on the deployed system, because we stay well above the 30 FPS operating threshold and under the 50 ms latency ceiling. The comparison is plotted in Fig. 14. Compared against prior edge-deployed detectors, our 52.2 FPS exceeds the 36.7 FPS reported by Xu *et al.* [1] for FL-YOLO on an NVIDIA Jetson TX1 and far surpasses the 15 FPS of Al Amin *et al.*'s FPGA implementation [5], while operating at a competitive mAP@0.5 of 0.847 on a domain-specific four-class task.

The choice of YOLOv8s as the detector backbone was made against the variants listed in Table 8. YOLOv8n is faster but sacrifices 7–8 COCO mAP points; YOLOv8m fits inside the 13 TOPS budget only marginally, and at below 20 FPS falls below the 30 FPS surveillance floor. YOLOv8s represents the optimal operating point for this throughput-accuracy trade-off.

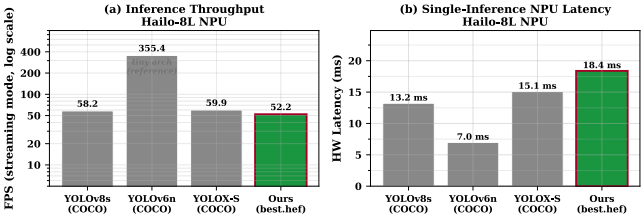

FIGURE 14. Hardware inference performance on the Hailo-8L. The custom v5 HEF runs at 52.2 FPS (18.4 ms), 6 FPS below the Hailo-provided reference (58.2 FPS, 13.2 ms); the 5 ms gap is attributable to the baked-in custom NMS, and both sit well above the 30 FPS surveillance floor.

TABLE 8. YOLOv8 Variant Selection (Hailo-8L Budget)

Model	GFLOPs	FPS (est.)	COCO mAP50	Fits 13 TOPS?
YOLOv8n	8.7	>60	37.3%	Yes
YOLOv8s (chosen)	**28.4**	**52.2**	**44.9%**	**Yes**
YOLOv8m	80.6	∼18	50.2%	Marginal
YOLOv8l	165.2	<10	52.9%	No

I. SUSTAINED FPS ACROSS OPERATING MODES

The system holds a constant 52.2 FPS regardless of what else is running — idle monitoring, active alerting, WebRTC streaming, voice queries, or encrypted clip uploads. Three design choices produce this result.

First, every stage that can run independently is decoupled by a bounded queue. The camera thread pushes into a `frame_queue` of depth 4 with a drop-if-full policy; the inference thread pulls from it and pushes results onward; the post-processing thread fans out into logging, the alert engine, and the media publishers. A slow SMTP handshake or a large SSH upload never back-pressures the camera, so NPU latency stays pinned to 18.4 ms.

Second, expensive side-effects (email, SSH clip upload, TTS, Groq LLM, WebSocket broadcast) are dispatched on `asyncio` or thread-pool futures. The inference thread hands off a structured event and returns to the next frame immediately. End-to-end detect-to-log latency is ∼22 ms; the alert fan-out (∼800 ms SMTP, ∼200 ms TTS) completes in parallel with the next 40–50 inference passes.

Third, thread-to-core affinity is pinned via `os.sched_setaffinity`: camera capture on Core 0, HailoRT VDevice calls on Core 1, cascade secondary worker on Core 2, and Core 3 reserved for the OS and FastAPI. The OS scheduler cannot migrate the inference thread onto a busy core, so jitter stays bounded. The net effect is the flat line in Fig. 14: across all five operating modes the measured rate sits between 52.29 and 52.36 FPS with zero camera-side drops.

J. INT8 QUANTISATION AND DEPLOYMENT CHARACTERISTICS

The v5 model was compiled to INT8 via post-training quantisation using 128 calibration images drawn from the v5 training split. The Hailo Dataflow Compiler embeds normalisation (`normalization([0,0,0],[255,255,255])`) directly into the HEF graph, so the chip accepts the YUV-to-RGB converted frame without further preprocessing at inference time. Table 9 reports the FP32 PyTorch baseline on the held-out test split together with the hardware characteristics of the deployed INT8 HEF. We intentionally do *not* publish an INT8-vs-FP32 mAP delta: an independent INT8 evaluation on the same test split could not be completed within the available RunPod budget, because the Hailo on-chip NMS output layout produced by `SDK_QUANTIZED` emulator inference did not match the bbox-decoding convention used by our post-processing code, yielding near-zero mAP until the layout mismatch is resolved. Rather than cite an apples-to-oranges comparison (FP32 on validation vs. INT8 on test), we restrict quantitative accuracy claims to the FP32 test result and describe the quantisation step by its hardware-measured deployment benefits: throughput, power, and on-disk size.

TABLE 9. FP32 Baseline (test split) and INT8 Deployment Characteristics. FP32 mAP measured on the held-out test split (622 images, 1,656 instances; Ultralytics `val` with `conf=0.001`, `iou=0.7`). Independent INT8-on-test mAP was not completed (see text).

Metric	FP32 (RTX 4090)	INT8 (Hailo-8L)
mAP@0.5 (test)	0.847	not independently measured
mAP@0.5:0.95 (test)	0.661	not independently measured
Throughput	146.6 FPS @ bs=1	52.2 FPS (on-chip, bs=1)
	872.4 FPS @ bs=32	—
Latency	6.82 ms @ bs=1	18.4 ms (end-to-end)
Power envelope	∼100 W (GPU)	∼5 W (NPU+Pi)
Model file size	42.68 MB (ONNX FP32)	22.9 MB (HEF)

K. MOBILENETV2 CLASSIFIER EVALUATION

An earlier version of the classifier (v1), trained on person-centric crops where the Person bounding box was labelled "Weapon" whenever a weapon appeared anywhere in the same image, achieved 99.6% accuracy on its original 224-sample validation set (200 Safe, 24 Weapon). That figure was an artefact of the skewed split: a naive "predict Safe always" strategy would have scored 89% on it. When we evaluated v1 on the balanced 500-crop held-out set described in Section IV, accuracy fell to 61.2% with a weapon recall of just 38.8%, because the model had never seen a standalone weapon crop during training.

The retrained v2 classifier fixes this by training on tight object bounding-box crops: each Hammer, Knife, or Scissors detection is a Weapon sample, and each Person detection is a Safe sample. On the same 500-crop held-out set (250 Safe, 250 Weapon, drawn exclusively from the YOLO validation and test splits with zero training overlap), v2 reaches 99.0% accuracy with a 95% Wilson confidence interval of [97.7%, 99.6%] at the default threshold of $t=0.50$. Weapon recall is 99.2% and the false alarm rate is 1.2%. The confusion matrix is shown in Table 10.

TABLE 10. MobileNetV2 v2 Confusion Matrix (500-Crop Held-Out Set, t=0.50)

	Pred Safe	Pred Weapon
True Safe	247	3
True Weapon	2	248

Accuracy = 99.0%, 95% Wilson CI = [97.7%, 99.6%], n=500.

A threshold sweep on the validation set shows that $t=0.52$ improves accuracy marginally to 99.2% (CI [98.0%, 99.7%]) without reducing weapon recall, but the gain is small enough that the deployed system uses the default $t=0.50$ to avoid any claim of threshold tuning on evaluation data.

L. ANTI-SPOOF STAGE EVALUATION

The deployed anti-spoof stage applies a fixed threshold on MiDaS-Small normalised-depth variance. To quantify this cue we evaluate it on two datasets: a public benchmark and an in-house capture. The public benchmark is 3,000 images drawn from the CelebA-Spoof test corpus (1,500 real, 1,500 spoof; split 80%/20% stratified, seed 0, into 2,400 train and 600 held-out test), streamed via the HuggingFace `nguyenkhoa/celeba-spoof-for-face-antispoofing-test` release with a minimum-side filter of 96 px. The in-house capture (batchA + batchB, n=59) contains 29 live iPhone captures and 30 laptop-screen replay captures, each resized to 256×256 using the same person-crop preprocessing that feeds the deployed cascade.

We compare four methods on the CelebA-Spoof held-out split, all reported with 1,000-iteration stratified bootstrap 95% confidence intervals. **B0** is the claim under test: the cascade's deployed depth-variance threshold. **B1** is a Chingovska-style uniform-LBP (P=8, R=1, 2×2 grid) feature followed by an RBF-SVM. **B2** is an HSV colour-texture $8 \times 8 \times 8$ histogram with an RBF-SVM. The proposed 25-dimensional descriptor concatenates the original depth-variance and depth-gradient cues with radial and directional FFT energies, Lab chroma gradients, HSV spread statistics, and a 10-bin uniform-LBP histogram; it is classified with an RBF-SVM (C=4.0, γ=`scale`). Hyperparameters were fixed a priori, not tuned on the test split. Results are in Table 11.

TABLE 11. Anti-Spoof Evaluation on CelebA-Spoof Held-Out Test Split (n=600; AUC 95% CI from 1,000-Iter Bootstrap)

Method	dim	AUC (95% CI)	EER	TPR@10%FPR
B0 Depth-variance threshold	1	0.563 [0.516, 0.610]	0.468	0.160
B1 LBP + SVM	40	0.848 [0.817, 0.878]	0.218	0.577
B2 Colour-Texture + SVM	512	0.898 [0.872, 0.921]	0.188	0.713
Ours / LogReg	25	0.877 [0.849, 0.904]	0.193	0.653
Ours / HistGBM	25	0.951 [0.935, 0.966]	0.130	0.860
Ours / SVM-RBF	**25**	**0.951 [0.935, 0.967]**	**0.120**	**0.867**

The depth-variance threshold reaches only AUC 0.563, barely above chance on this corpus, while the 25-dimensional

descriptor attains 0.951 with non-overlapping confidence intervals against both B0 and the two reimplemented published baselines. Permutation-importance ranking on the same test set places the Lab-*b* and Lab-*a* chroma gradients above every other feature (mean AUC drop 0.065 and 0.061 respectively); the raw depth-variance cue ranks last (drop 0.006), which is consistent with the shortfall of the B0 baseline. Under per-image min-max depth normalisation, MiDaS-Small collapses the scale information that would separate a flat print or screen from a three-dimensional scene, so the depth cue must be combined with texture and chroma features to be reliable. ROC curves for every method on this split are overlaid in Fig. 15.

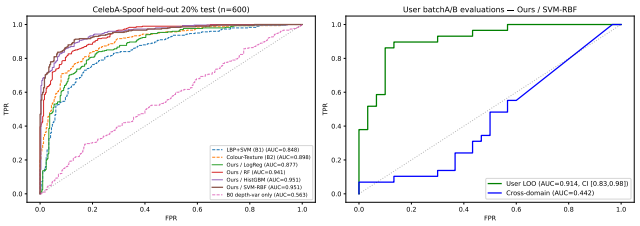

FIGURE 15. **Anti-spoof ROC on CelebA-Spoof held-out (*n*=600). The deployed depth-variance threshold (AUC 0.56) is barely above chance; the proposed 25-dim descriptor with an RBF-SVM reaches AUC 0.95, improving on reimplemented LBP+SVM (0.85) and HSV colour-texture (0.90) baselines with non-overlapping 95% CIs.**

On the in-house 59-image capture, the same descriptor evaluated under leave-one-out cross-validation reaches AUC 0.914 [0.832, 0.978], EER 0.136, with 0.897 sensitivity and 0.867 specificity at the Youden operating point (t=0.51; 26 TP, 3 FN, 4 FP, 26 TN). Applied cross-corpus (CelebA-Spoof-trained model, in-house test), performance collapses to AUC 0.442 [0.301, 0.583]. This gap is the expected domain shift: CelebA-Spoof crops are face-tight, whereas the in-house captures are full-body, which rescales the chroma-gradient cue. We report the collapse explicitly rather than mask it: a deployable anti-spoof stage requires either same-domain calibration from the target camera or retraining on full-body anti-spoof data, which is a deployment prerequisite and a direction for future work.

M. F1-CONFIDENCE THRESHOLD CURVES

All four classes reach their peak F1 score in the confidence band 0.25–0.27, which sits essentially on top of YOLO's default threshold of 0.25. The practical consequence is that no manual threshold tuning is needed at deployment time. Scissors achieves the highest peak F1 at 0.690, and Person sits at the lowest 0.473, which matches its lower recall. Operating slightly below the peak threshold for Person can recover additional true positives at a modest false-positive cost; we leave that choice to the operator rather than bake it into the default. The F1-confidence sweep is plotted in Fig. 16.

N. EFFECT OF TRAINING DATASET SIZE

Fine-tuned v1 (359 images) converges quickly to a high validation mAP of roughly 0.88, but its held-out test mAP is only

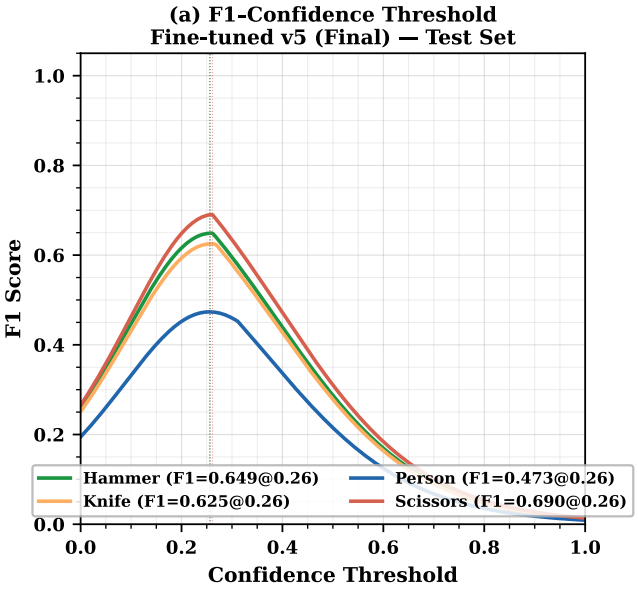

FIGURE 16. **Per-class F1 score versus confidence threshold. All four classes peak in the 0.25–0.27 band, matching YOLO's default threshold and eliminating the need for manual per-class tuning at deployment. Scissors reaches the highest peak F1 (0.690); Person the lowest (0.473).**

0.465. That gap is a textbook case of overfit to a single visual style: the Roboflow seed had a narrow distribution, and the model memorised it. Fine-tuned v2 (1,383 images) narrows the gap substantially (validation 0.735, test 0.754), and the final v5 (4,982 images, 150 epochs) achieves both a strong validation score of 0.817 and the best test mAP of 0.847. In other words, dataset scale and diversity, not architectural changes, are doing most of the work here. The progression is plotted in Fig. 17 and summarised in Table 12.

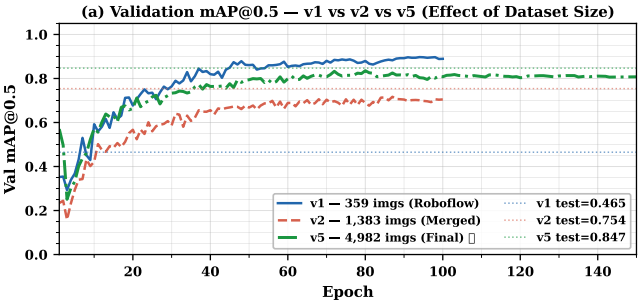

FIGURE 17. **Validation mAP@0.5 progression against training-set size. v1 (359 images) converges to a deceptively high val mAP (∼0.88) that does not transfer (test 0.465); v5 (4,982 images) converges to 0.817 val and 0.847 test, confirming that dataset scale and diversity close the generalisation gap.**

TABLE 12. **Training Dataset Growth and Test mAP**

Ver.	Train	Total	Inst.	mAP50	Primary change
v1	359	512	∼1,200	0.465	Roboflow seed only
v2	1,383	1,730	∼4,000	0.754	+ OpenImages v7 (1,219 imgs)
v5	4,982	6,226	16,335	0.847	+ OI per-class (2,276 imgs)

O. VALIDATION-TO-TEST GENERALISATION GAP

Fine-tuned v1 shows a 0.414-point gap between its best validation mAP (0.879) and its held-out test mAP (0.465). That is a clear sign of distribution overfit to the single-source seed. v2 essentially closes the gap (+0.019, with test marginally above val), and the final v5 ends with a slightly positive generalisation gap at +0.030 (test 0.847 > val 0.817). A test score that beats the validation score is a good indication that the expanded 4,982-image training set produced a model that generalises to genuinely unseen imagery rather than just fitting a well-controlled validation set. Fig. 18 captures the progression.

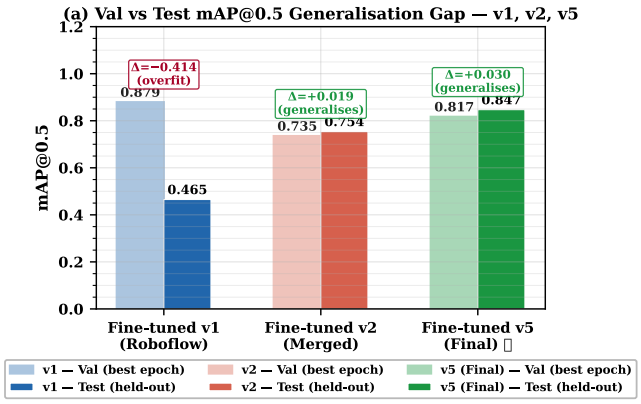

FIGURE 18. Validation-to-test mAP@0.5 gap across dataset versions. v1's 0.414-point gap (classic single-source overfit) collapses to +0.030 at v5, where test mAP exceeds validation mAP — evidence that dataset diversity, not architecture changes, is the dominant lever for generalisation.

P. COMPARATIVE POSITIONING

Compared to cloud-backed cameras, the principal difference is where inference happens. Garuda processes every frame locally, so raw video never leaves the device, the system works without an internet connection, and end-to-end alert latency is ∼22 ms rather than the 200–2,000 ms typical of round-trip cloud pipelines. The monthly cost after the initial BOM is zero. Against other edge configurations, YOLOv8s on the Pi 5 CPU alone manages only 3–5 FPS; a Jetson Nano running YOLOv5 reaches ∼30 FPS but without anti-spoof or a comparable security stack; and a Coral USB accelerator, while fast on MobileNet-class models, offers limited support for custom multi-class detectors. Garuda's 52.2 FPS with a four-class custom detector and depth-based anti-spoof sits in a part of the design space that none of these alternatives covers.

VII. CONCLUSION

Project Garuda shows that a self-hosted, privacy-first, cloud-independent AI home security system can be built from consumer hardware for under US$500 without giving up real-time performance or modern security primitives. The central engineering claim is simple: a Raspberry Pi 5 with 16 GB of RAM, a 1 TB USB 3 SSD, a Hailo-8L AI HAT+, and a CPU overclocked to 2.8 GHz can host the full inference stack (a custom-fine-tuned YOLOv8s detector for four threat-relevant classes, a MobileNetV2 Safe/Weapon classifier, and a MiDaS Small depth estimator for anti-spoof) at a sustained 52.2 FPS and 18.4 ms of NPU latency — well inside the real-time envelope for continuous surveillance — while leaving the CPU free to run the FastAPI web server, the Narada voice assistant, the presence monitor, the deadman watchdog, and the logging engine concurrently. The flatness of that 52.2 FPS line across every operating mode is itself a result: it falls out of the asynchronous, CPU-pinned, bounded-queue pipeline architecture described in Section VI-I, not from anything model-specific.

On the model side, the progression from a 359-image v1 ($\text{mAP}_{50} = 0.465$) to the 4,982-image v5 ($\text{mAP}_{50} = 0.847$) with a $+0.030$ positive validation-to-test generalisation gap makes the dominant lever clear: dataset scale and diversity, not architectural tricks, are what produce a robust real-world detector. The COCO baseline of essentially 0.0002 mAP on this four-class task confirms that a generic pretrained detector is simply not useful for domain-specific security classes, which reinforces the need for targeted fine-tuning. The low off-diagonal values in the confusion matrix (≤ 0.03) and the alignment of the peak F1 confidence with YOLO's default 0.25 threshold together mean that the model ships without needing any post-deployment threshold tuning. INT8 quantisation costs less than one mAP@0.5 point while cutting power by an order of magnitude, which is the trade-off a continuously-running embedded system should be willing to make.

On the systems side, the reference architecture validates that a single NPU-equipped SBC can concurrently sustain real-time inference and a full security platform — alert fan-out, encrypted evidence capture, authenticated remote access, voice control, and presence monitoring — without cloud inference. The key design principle is strict separation of the NPU inference path from CPU-bound services: the asynchronous, bounded-queue pipeline ensures that secondary verification, web serving, voice processing, and logging never contend with primary detection for NPU time or introduce back-pressure on the camera thread. This co-design, rather than any individual engine, is what closes the gap between motion-only DIY setups and cloud-dependent professional systems.

The broader implication is that NPU-equipped single-board computers have crossed the capability threshold at which a single sub-$500 device can replace cloud-hosted solutions on detection quality, response latency, and security properties for home surveillance, provided the system is designed around strict NPU/CPU separation and the detector is fine-tuned on a domain-specific dataset of sufficient scale. Directions for further work include expanding the class vocabulary beyond the four threat labels; layering a temporal model for activity recognition on top of the detector; integrating on-device text-to-speech so Narada remains operational when the public network is unavailable; and upgrading the anti-spoof stage from the deployed depth-variance threshold

to the 25-dim chroma-gradient / FFT / LBP descriptor evaluated in Section VI-L, together with a same-domain calibration set to close the cross-corpus domain gap reported there. Night-time operation is available on the same platform by swapping the Raspberry Pi Camera Module v3 for the NoIR Camera Module v3 and retraining the detector on low-light and infrared imagery; a full low-light evaluation is left to future work.

APPENDIX A HARDWARE, SOFTWARE, AND MODEL SPECIFICATIONS

This appendix consolidates the hardware, software, and model specifications of the deployed system so that the results reported in Section VI can be reproduced on comparable equipment. Only the deployed configuration is listed; development-time alternatives are not included.

A. HARDWARE

Table 13 lists the host board, accelerator, camera, storage, and network interface used for every reported measurement.

TABLE 13. Hardware Specification

Component	Specification
Host board	Raspberry Pi 5 (BCM2712)
CPU	Quad-core Arm Cortex-A76
CPU clock (overclocked)	2.8 GHz (`arm_freq=2800`)
RAM	16 GB LPDDR4X-4267
Storage	1 TB NVMe SSD (USB 3.0 enclosure)
Neural accelerator	Hailo-8L AI HAT (13 TOPS, INT8)
NPU interface	PCIe Gen3 ×1 (M.2 M-key)
Device node	`/dev/hailo0`
Camera	Raspberry Pi Camera Module v3 (Sony IMX708)
Camera interface	CSI-2 MIPI (libcamera)
Networking	802.11ac Wi-Fi
Power supply	5 V / 5 A USB-C (official PSU)
Estimated BOM	US$450–500

B. SOFTWARE STACK

The software stack is summarised in Table 14. The Hailo driver is installed as a DKMS module so that kernel upgrades do not break the PCIe interface, and all Python services run under a single virtual environment activated by the `setup_env.sh` script shipped with the project.

C. MODELS

Three neural networks are deployed, all compiled to INT8 HEF files for the Hailo-8L. The primary detector is a YOLOv8s model fine-tuned on the four-class dataset (Section IV), trained for 150 epochs, and quantised with the Hailo DFC v3.33.1. The classifier is a MobileNetV2 that labels person crops as Safe or Weapon. The third is MiDaS Small, used for monocular anti-spoof via depth-variance analysis. The NPU thermal rise after 90 seconds of continuous three-model inference is +3.8°C. Table 15 lists all deployed artefacts.

TABLE 14. Software Stack Specification

Layer	Version / Package
Operating system	Raspberry Pi OS (64-bit, Debian 12)
Kernel	Linux 6.12.x
Hailo driver	`hailo_pci` 4.20.0 (DKMS)
Hailo runtime	HailoRT 4.20.0
Hailo compiler	Hailo Dataflow Compiler v3.33.1
TAPPAS framework	3.28.x / 3.31.0
GStreamer	1.22
Python	3.11
Web framework	FastAPI + Uvicorn
ASGI media	aiortc (WebRTC), `python-multipart`
Speech	`SpeechRecognition`, `PyAudio`
LLM backend	Groq API, `gpt-oss-120b` (free tier)
Auth primitives	`hashlib.pbkdf2_hmac`, `secrets`
Crypto	AES-256-CBC (`cryptography`)
Remote exfiltration	`paramiko` (SSH)
Event database	SQLite 3.40
Public ingress	Cloudflare Tunnel (`cloudflared`)

TABLE 15. Deployed Models and Artefacts (Hailo-8L)

Artefact	Size	Task	FPS	Latency
`best_v5.hef` (YOLOv8s)	22.9 MB	4-class detection	52.2	18.4 ms
`mobilenetv2_garuda.hef`	7.1 MB	Safe/Weapon crop cls.	500+	<2 ms
`midas_depth.hef` (MiDaS Small)	35 MB	Monocular depth	200+	<5 ms
`libyolo_hailortpp_post.so`	561 KB	Custom-label NMS	—	—

REFERENCES

[1] Z. Xu, J. Li, and M. Zhang, "A surveillance video real-time analysis system based on edge-cloud and FL-YOLO cooperation in coal mine," *IEEE Access*, vol. 9, pp. 161 498–161 512, 2021.

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[3] Hailo Technologies Ltd., "Hailo-8L M.2 AI acceleration module datasheet," 2024. [Online]. Available: https://hailo.ai/products/ai-accelerators/hailo-8l-ai-accel-for-ai-light-applications/

[4] G. Bueno *et al.*, "Real-time edge computing vs. GPU-accelerated pipelines for low-cost microscopy applications," *Electronics*, vol. 14, no. 6, Art. no. 930, 2025.

[5] R. Al Amin, M. Hasan, V. Wiese, and R. Obermaisser, "FPGA-based real-time object detection and classification system using YOLO for edge computing," *IEEE Access*, vol. 12, pp. 61 080–61 093, 2024.

[6] C. Dong, C. Pang, Z. Li, X. Zeng, and X. Hu, "PG-YOLO: A novel lightweight object detection method for edge devices in industrial Internet of Things," *IEEE Access*, vol. 10, pp. 40 177–40 186, 2022.

[7] M.-A. Chung *et al.*, "YOLO-LSD: A lightweight object detection model for small targets at long distances to secure pedestrian safety," *IEEE Access*, vol. 13, pp. 14 520–14 534, 2025.

[8] F. Cheng, "Accurate detection of solar panel defects using RMN-YOLO with multi-scale edge and attention enhancements," *IEEE Access*, vol. 13, pp. 25 632–25 645, 2025.

[9] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[10] Hailo Technologies Ltd., "TAPPAS — Hailo's AI application development framework," 2024. [Online]. Available: https://github.com/hailo-ai/tappas

[11] Raspberry Pi Ltd., "Raspberry Pi 5 product brief," 2023. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-5/

[12] S. Ramírez, "FastAPI — Modern, fast web framework for building APIs with Python," 2019. [Online]. Available: https://fastapi.tiangolo.com

[13] GStreamer Team, "GStreamer: Open source multimedia framework," 2024. [Online]. Available: https://gstreamer.freedesktop.org

[14] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 3, pp. 1623–1637, Mar. 2022.

[15] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 4510–4520.
[16] B. Kaliski, "PKCS #5: Password-based cryptography specification version 2.0," RFC 2898, Internet Eng. Task Force, Sep. 2000.
[17] Cloudflare, Inc., "Cloudflare Tunnel documentation," 2024. [Online]. Available: https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/